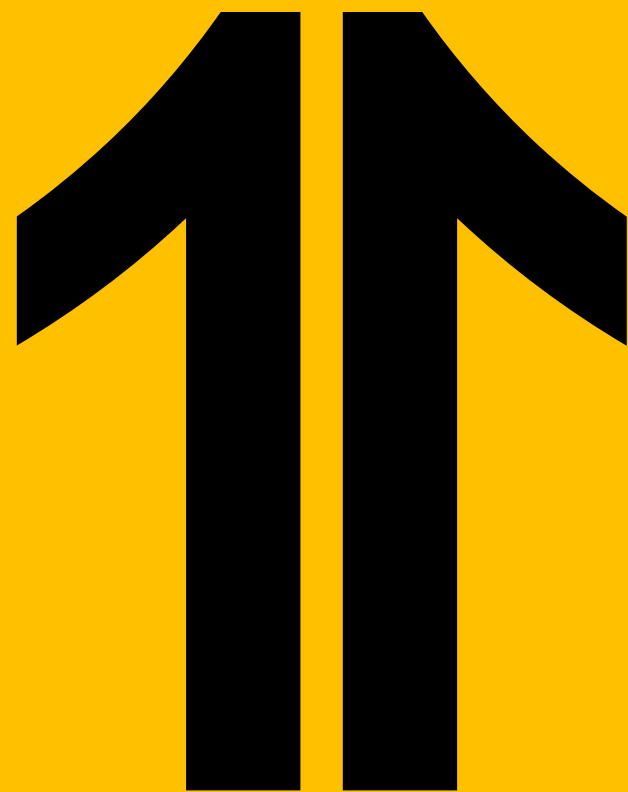




UPN

**UNIVERSIDAD
PRIVADA
DEL NORTE**



ESTRUCTURA DE DATOS

Listas dobles



LOGRO DE LA SESIÓN

Al término de la clase, el estudiante crea y maneja listas enlazadas dobles en C# mostrando dominio técnico del lenguaje, y una lógica coherente.



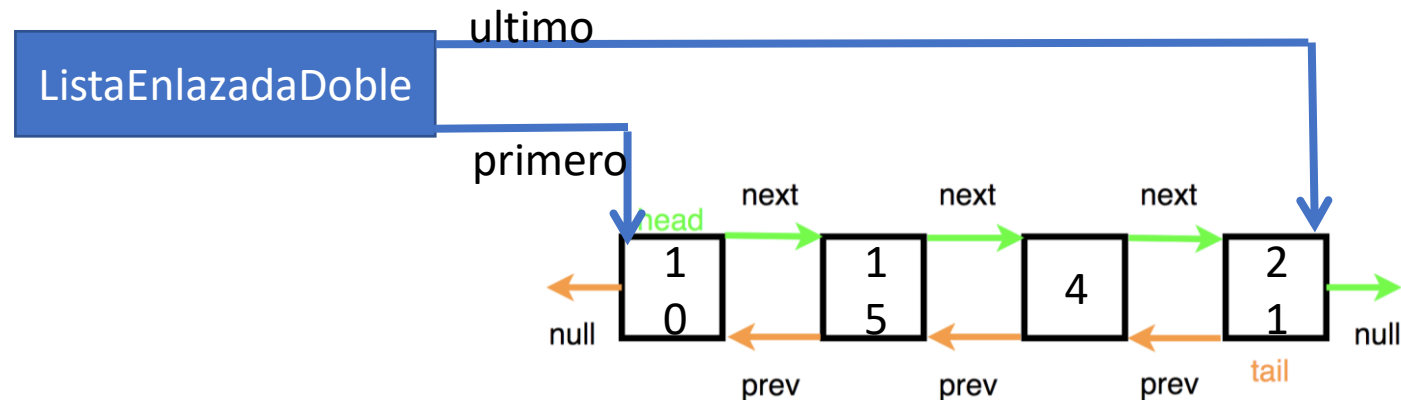
¿Qué son listas
enlazadas
dobles?



LISTA DOBLEMENTE ENLAZADA



Una lista doble es una estructura donde el último elemento tiene como referencia siguiente null y la referencia al anterior del primer elemento de la lista también es null. Cada elemento está doblemente enlazado.

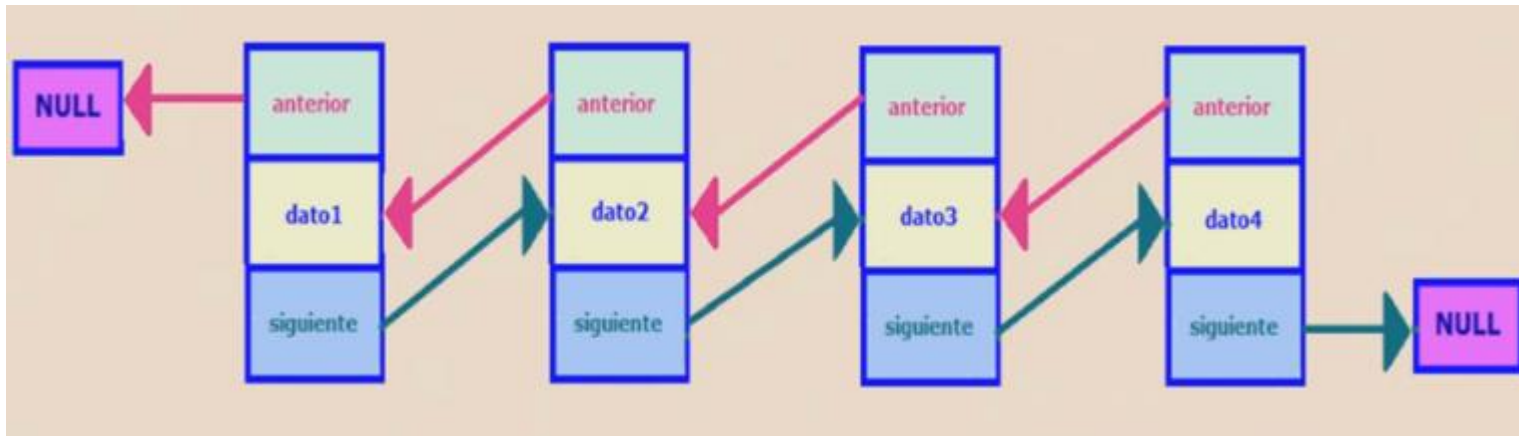


A través del uso de listas dobles podemos acceder a los datos recorriéndolos hacia delante hasta el final o hacia atrás hasta el inicio.

LISTA DOBLEMENTE ENLAZADA



Es una lista lineal en la que cada nodo tiene dos enlaces. Uno al nodo siguiente y otro al anterior, además de guardar un dato.



Hay que tener en cuenta que el requerimiento de memoria es mayor en las listas doblemente encadenadas ya que tenemos dos punteros por nodo. La estructura del nodo es:

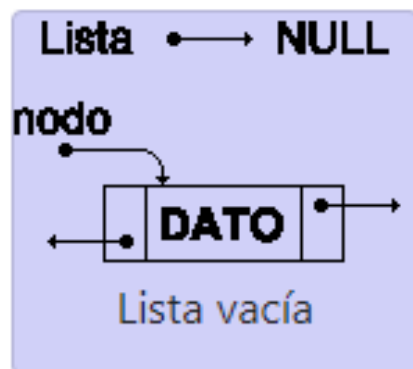
```
class Nodo {  
    public int info;  
    public Nodo sig, ant;  
}
```

OPERACIONES CON UNA LISTA DOBLEMENTE ENLAZADA



- ☐ Creación de la lista
- ☐ Inserción de un dato al inicio/al final
- ☐ Eliminación de un dato
- ☐ Recorridos (en ambos sentidos)
- ☐ Búsqueda
- ☐ Eliminar todos los nodos (vaciar)

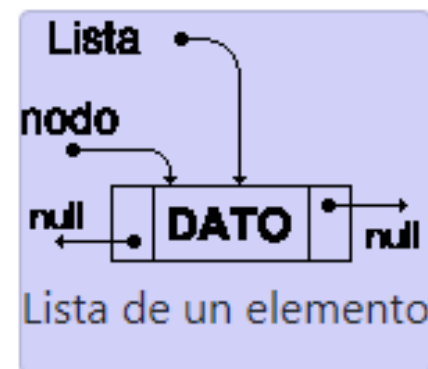
Añadir elemento en una lista doblemente enlazada vacía



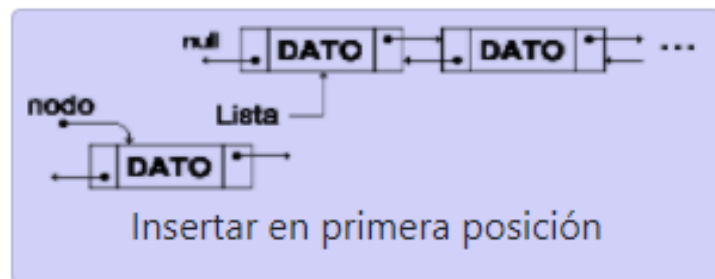
Partiremos de que ya tenemos el nodo a insertar y, por supuesto un puntero que apunte a él, además el puntero que define la lista, que valdrá NULL:

El proceso es muy simple, bastará con que:

1. lista apunta a **nodo**.
2. lista->siguiente y lista->anterior apunten a null.

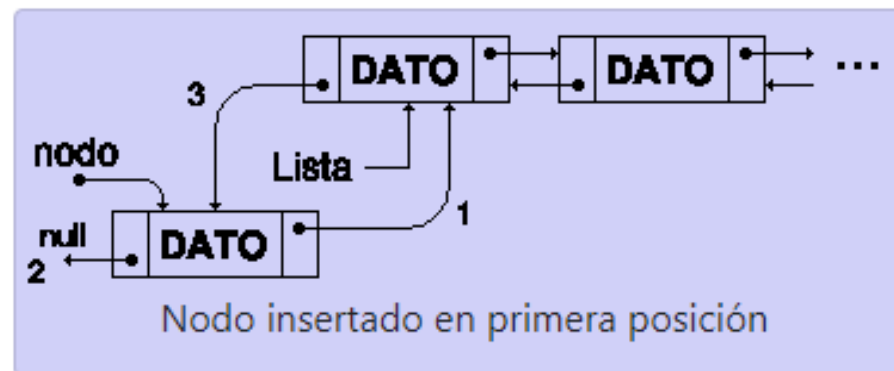


Insertar un elemento en la primera posición de la lista



Partimos de una lista no vacía. Para simplificar,

consideraremos que lista apunta al primer elemento de la lista doblemente enlazada:



El proceso es el siguiente:

1. **nodo**->siguiente debe apuntar a Lista.
2. **nodo**->anterior apuntará a Lista->anterior.
3. Lista->anterior debe apuntar a **nodo**.

Recuerda que Lista no tiene por qué apuntar a ningún miembro concreto de una lista doblemente enlazada, cualquier miembro es igualmente válido como referencia.

Añadir elemento en una lista doblemente enlazada, caso general



Para generalizar todos los casos anteriores, sólo necesitamos añadir una operación:

1. Si lista está vacía hacemos que Lista apunte a nodo. Y nodo->anterior y nodo->siguiente a NULL.
2. Si lista no está vacía, hacemos que nodo->siguiente apunte a Lista->siguiente.
3. Después que Lista->siguiente apunte a nodo.
4. Hacemos que nodo->anterior apunte a Lista.
5. Si nodo->siguiente no es NULL, entonces hacemos que nodo->siguiente->anterior apunte a nodo.

El paso 1 es equivalente a insertar un nodo en una lista vacía.

Los pasos 2 y 3 equivalen a la inserción en una lista enlazada corriente.

Los pasos 4, 5 equivalen a insertar en una lista que recorre los nodos en sentido contrario.

Existen más casos, las listas doblemente enlazadas son mucho más versátiles, pero todos los casos pueden reducirse a uno de los que hemos explicado aquí.

Eliminar un elemento de una lista doblemente enlazada

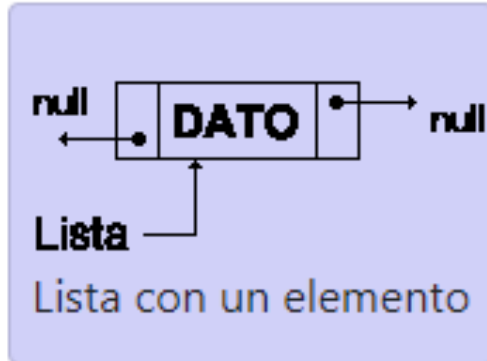


Analizaremos tres casos diferentes:

1. Eliminar el único nodo de una lista doblemente enlazada.
2. Eliminar el primer nodo.
3. Eliminar el último nodo.
4. Eliminar un nodo intermedio.

Para los casos que lo permitan consideraremos dos casos: que el nodo a eliminar es el actualmente apuntado por Lista o que no.

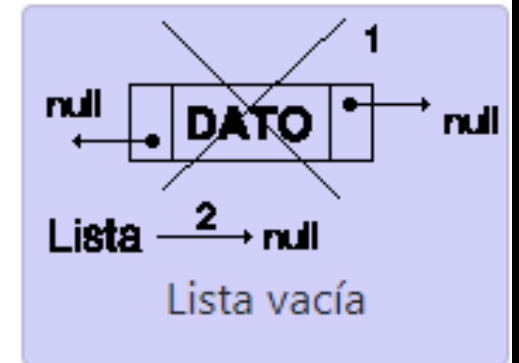
Eliminar el único nodo en una lista doblemente enlazada



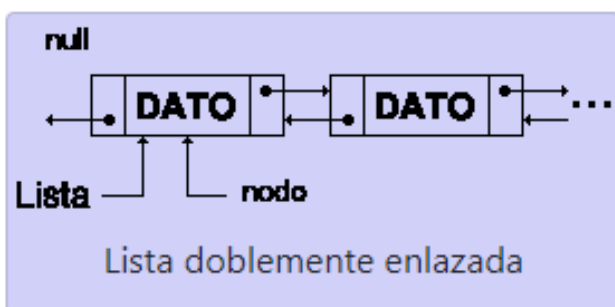
En este caso, ese nodo será el apuntado por Lista.

El proceso es simple:

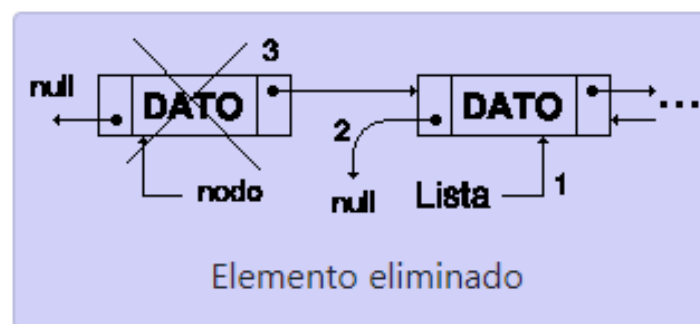
1. Eliminamos el **nodo**.
2. Hacemos que Lista apunte a NULL.



Eliminar el primer nodo de una lista doblemente enlazada



Tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que Lista sea Lista-



>siguiente.

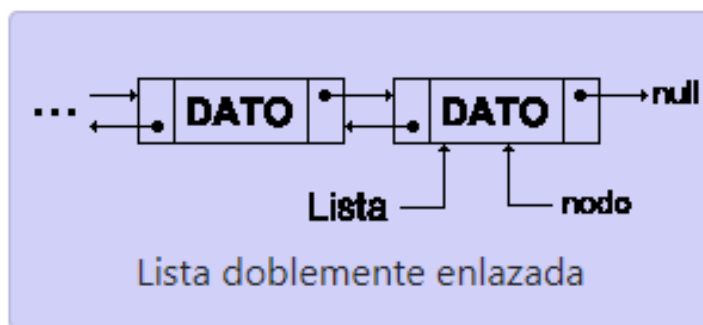
1. Si **nodo** apunta a Lista, hacemos que Lista apunte a Lista->siguiente.
2. Hacemos que **nodo**->siguiente->anterior apunte a NULL
3. Borramos el nodo apuntado por **nodo**.

El paso 2 separa el nodo a borrar del resto de la lista, independientemente del nodo al que apunte Lista.

Eliminar el último nodo de una lista doblemente enlazada



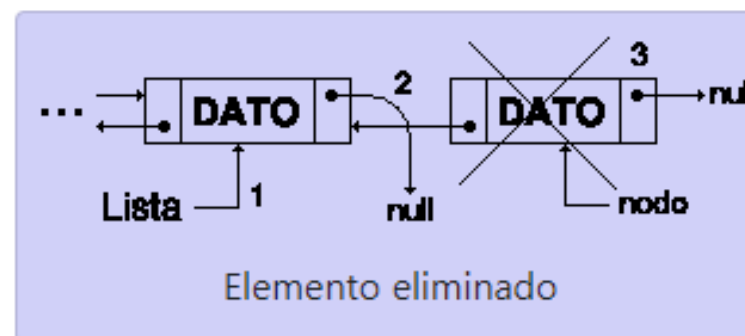
De nuevo tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que Lista sea Lista->anterior.



1. Si **nodo** apunta a Lista, hacemos que Lista apunte a Lista->anterior.
2. Hacemos que **nodo**->anterior->siguiente

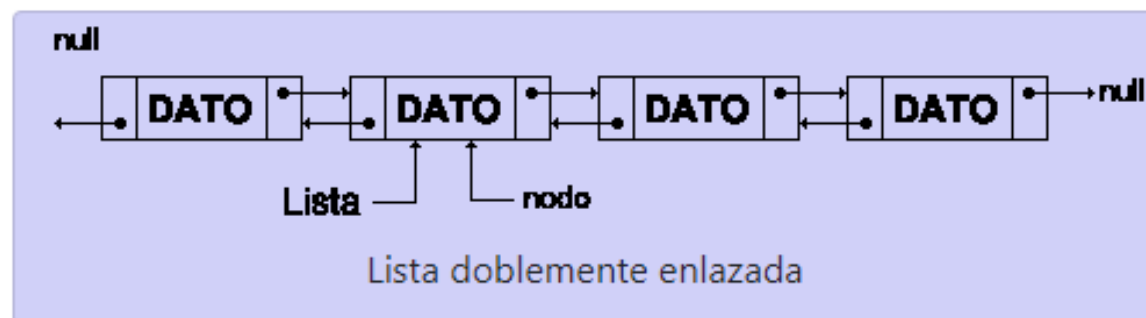
apunte a NULL

3. Borramos el nodo apuntado por **nodo**.



El paso 2 prepara el nodo a borrar del resto de la lista, independientemente del nodo al que apunte Lista.

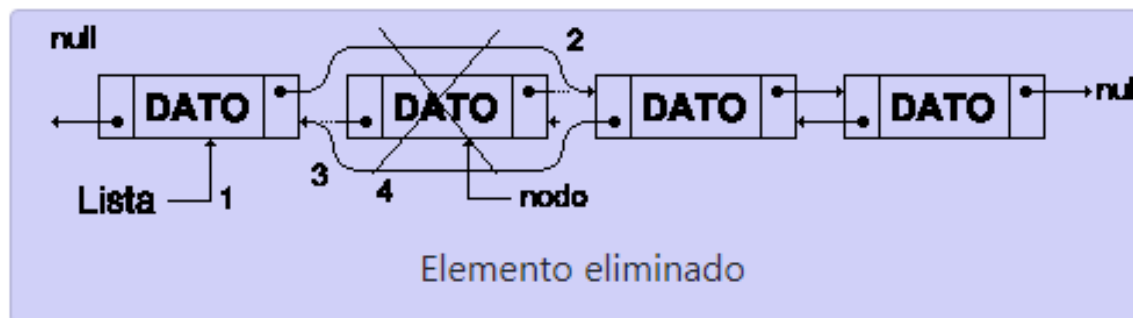
Eliminar un nodo intermedio de una lista doblemente enlazada



De nuevo tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que Lista sea Lista->anterior o Lista->siguiente

Se trata de un caso más general de los dos casos anteriores.

1. Si **nodo** apunta a Lista, hacemos que Lista apunte a Lista->anterior (o Lista->siguiente).
2. Hacemos que **nodo**->anterior->siguiente apunte a **nodo**->siguiente.
3. Hacemos que **nodo**->siguiente->anterior apunte a **nodo**->anterior.
4. Borramos el nodo apuntado por nodo.



Eliminar un nodo de una lista doblemente enlazada, caso general



De nuevo tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que Lista sea Lista->anterior, si no es NULL o Lista->siguiente en caso contrario.

1. Si **nodo** apunta a Lista,

- Si Lista->anterior no es NULL hacemos que Lista apunte a Lista->anterior.
- Si Lista->siguiente no es NULL hacemos que Lista apunte a Lista->siguiente.
- Si ambos son NULL, hacemos que Lista sea NULL.

2. Si **nodo**->anterior no es NULL, hacemos que **nodo**->anterior->siguiente apunte a **nodo**->siguiente.

3. Si **nodo**->siguiente no es NULL, hacemos que **nodo**->siguiente->anterior apunte a **nodo**->anterior.

4. Borraremos el nodo apuntado por **nodo**



Pozo, S. (s/f). *C Con Clase*. ConClase.net. Recuperado el 11 de marzo de 2022, de <https://conclase.net/c/edd/cap5>

Problemas resueltos de Listas. (s/f). Uabcs.mx. Recuperado el 11 de marzo de 2022, de http://uabcs.mx/difusion2017/files/libros/pdf/193_20160910055537.pdf

Nieves, C., Guerra, O., Martel, E. A., Carlos, J., Ramírez Casañas, M., Angel, M., & Suárez, Q. (s/f). Ulpqc.es. Recuperado el 11 de marzo de 2022, de <https://accedacris.ulpgc.es/bitstream/10553/1542/1/580.pdf>

http://www.aliat.org.mx/BibliotecasDigitales/sistemas/Estructura_de_datos/Estructura_de_datos_Parte_2.pdf



UPN

**UNIVERSIDAD
PRIVADA
DEL NORTE**